

Disciplina de Sistemas Operacionais

Trabalho Nº 3

System Design da Memória

em Sistemas Operacionais

Relação entre hardware, processador, MMU,
kernel, sistema operacional e processos

Gean Feitosa Cavalcante

Contents

Sumário do trabalho

Estrutura dos diagramas	
Padrão visual	

Sumário do trabalho

Este trabalho apresenta um **System Design da Memória em Sistemas Operacionais**, organizando visual e tecnicamente como a memória é utilizada por hardware, CPU, MMU, kernel, sistema operacional e processos — com foco prático no Linux moderno.

A entrega é composta por:

- **11 diagramas visuais**, seguindo paleta de cores e convenções definidas no escopo, cobrindo todos os artefatos obrigatórios e seções de conteúdo.
- **Texto de apoio**, explicando decisões do projeto, conceitos, relação entre visual e teoria, e resposta à pergunta central.
- **Referências**, com fontes oficiais (kernel.org, man7) e artigos acadêmicos.

Estrutura dos diagramas

#	Diagrama	Artefato obrigatório
1	Diagrama Macro da Arquitetura	Artefato 1
2	Hierarquia de Memórias	Seção 2
3	Fluxo de Tradução de Endereço	Artefato 2
4	Mapa do Espaço de Memória do Processo	Artefato 3
5	Proteção e Isolamento	Artefato 5
6	Técnicas de Gerenciamento (Comparativo)	Artefato 4
7	Fragmentação Interna vs Externa	Seção 7
8	Paginação e Memória Virtual	Seção 8
9	Linux na Prática	Artefato 6
10	Observabilidade e Monitoramento	Artefato 7
11	Conclusão Crítica Visual	Artefato 9

Padrão visual

Cor	Aplicação
Azul	Hardware e CPU
Roxo	MMU, TLB e tradução
Verde	Processo, aplicação e espaço do usuário
Vermelho	Proteção, falha, violação e fragmentação
Laranja	Kernel e subsistema de memória
Cinza	Armazenamento secundário e swap

Diagrama Macro da Arquitetura da Memória

Hardware → CPU → MMU/TLB → Kernel → Gerência de Memória do SO → Processos → Ferramentas de Observação

MODO KERNEL / HARDWARE



MODO USUÁRIO / USERSPACE

Legenda — Padrão Visual do System Design

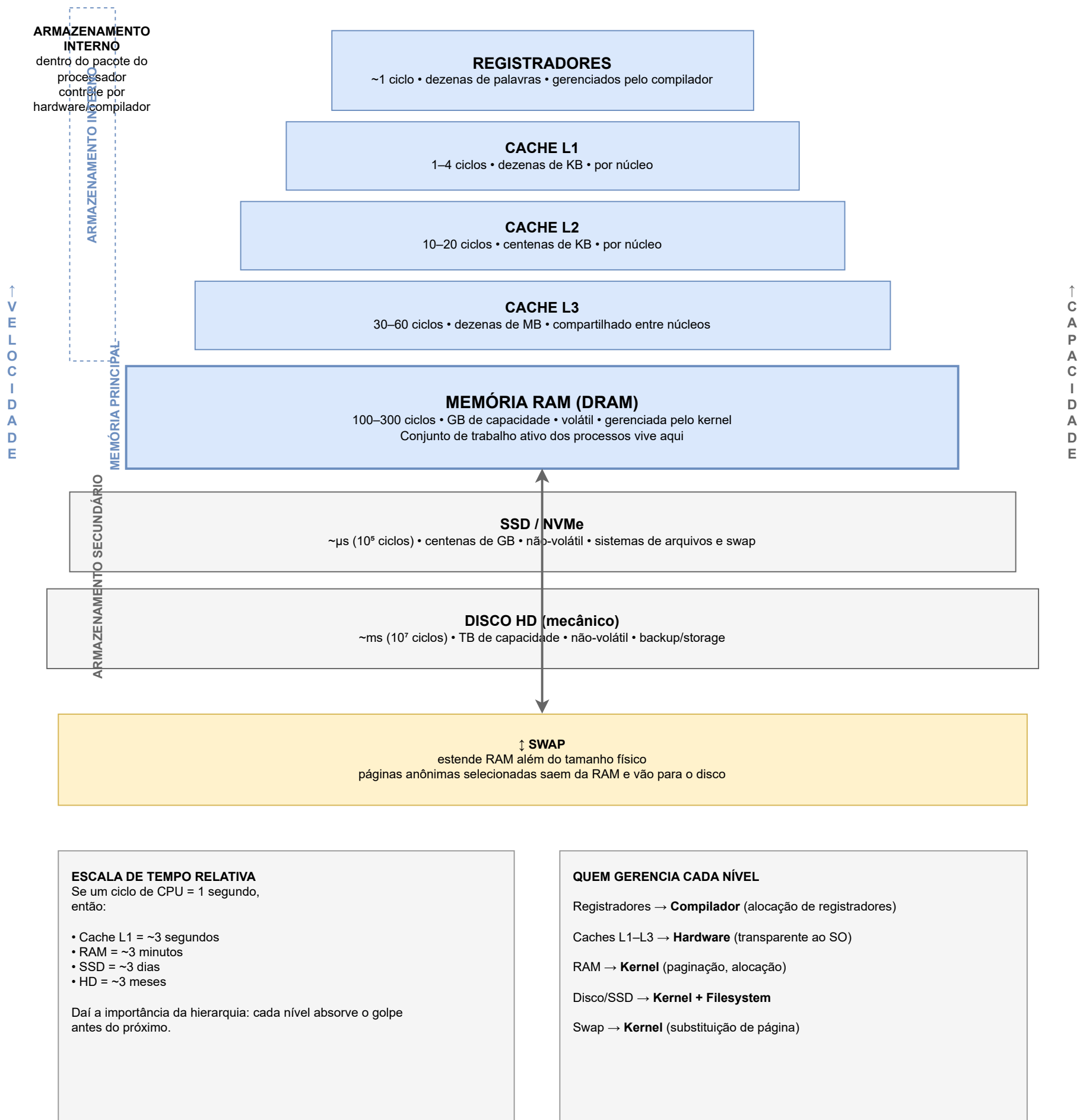
Azul — Hardware / CPU	Verde — Processo / userspace
Roxo — MMU / TLB / tradução	Vermelho — Proteção / falha / fragmentação
Laranja — Kernel / subsistema de memória	Cinza — Armazenamento secundário / swap

Perguntas-chave respondidas neste diagrama:

- Quem acessa memória? **CPU e processos**
- Quem traduz endereço? **MMU (com TLB)**
- Quem protege? **MMU + Kernel**
- Quem aloca? **Kernel / Gerência VM**
- Quem monitora? **Ferramentas Linux**

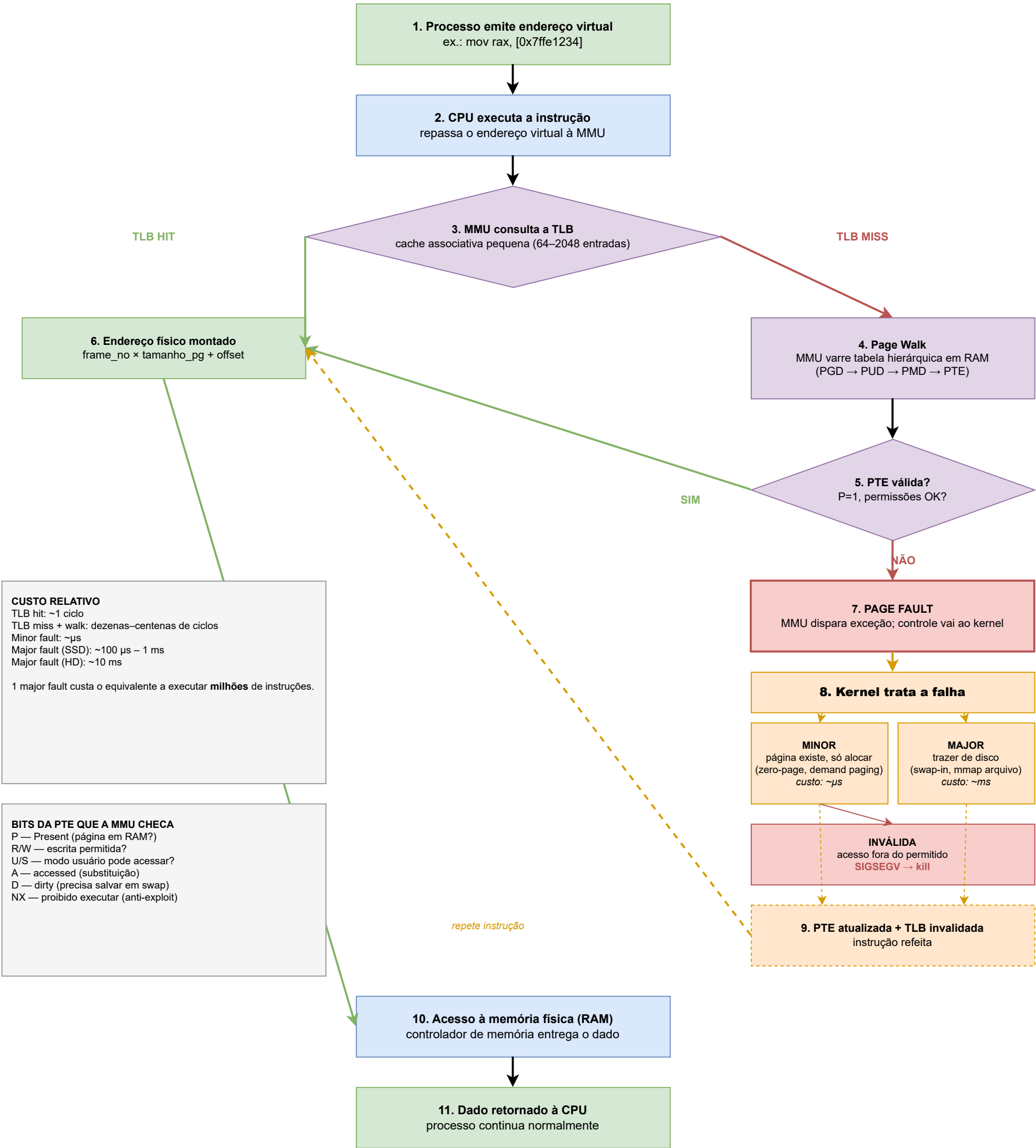
Classificação das Memórias — Hierarquia

Mais próximo da CPU → mais rápido, menor, mais caro | Mais distante → mais lento, maior, mais barato



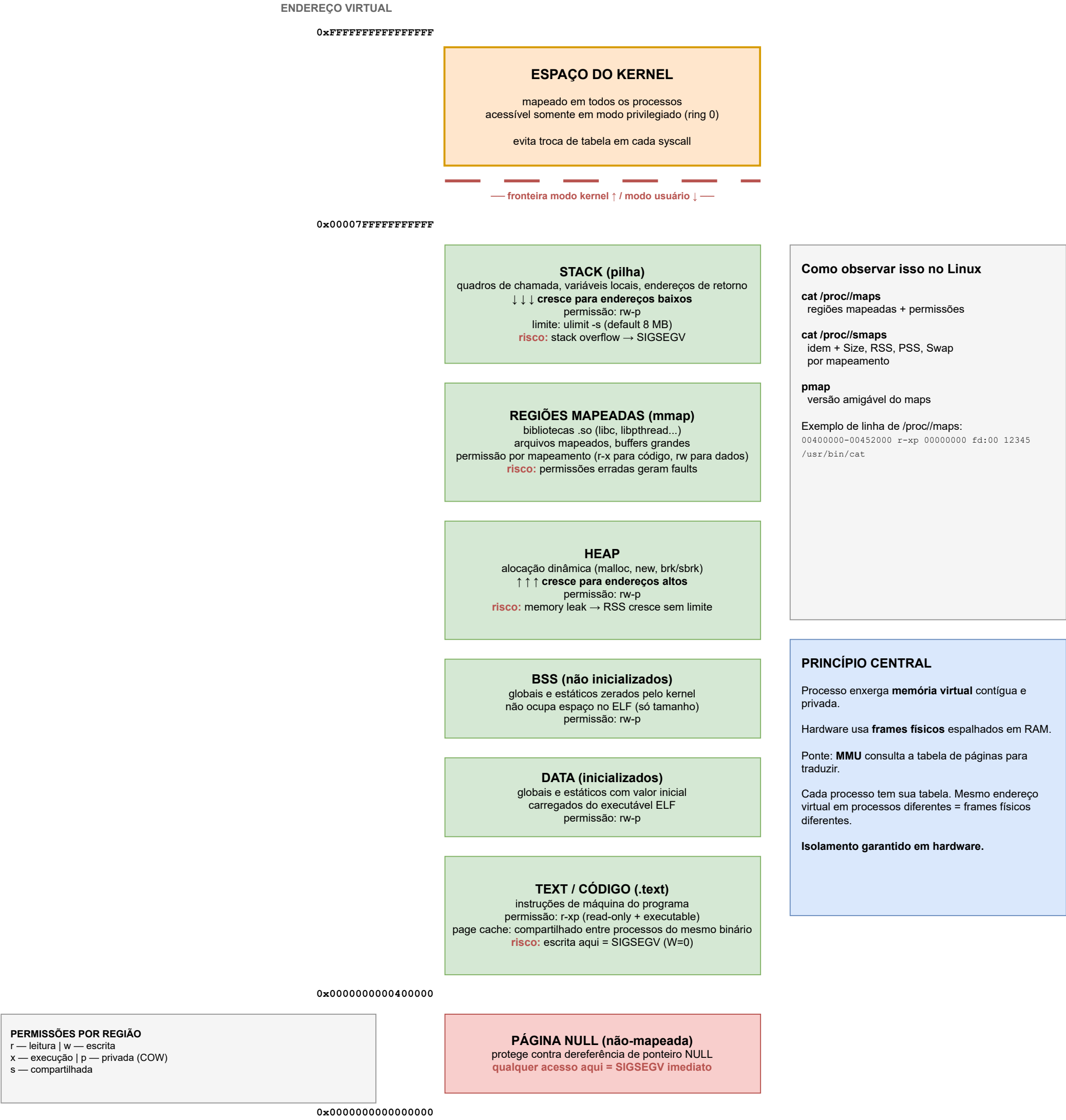
Fluxo de Tradução de Endereço Virtual → Físico

O que acontece quando um processo acessa um dado em memória?



Mapa do Espaço de Memória de um Processo (Linux x86-64)

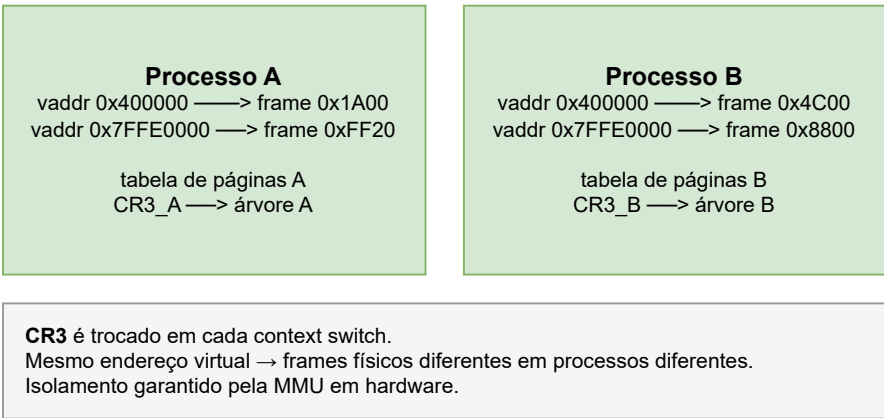
O processo enxerga memória de forma virtual e contígua. O kernel mapeia para frames físicos arbitrários.



Proteção e Isolamento de Memória

Os três pilares: espaços virtuais separados — permissões por página — modos de execução

1. ESPAÇOS VIRTUAIS SEPARADOS POR PROCESSO

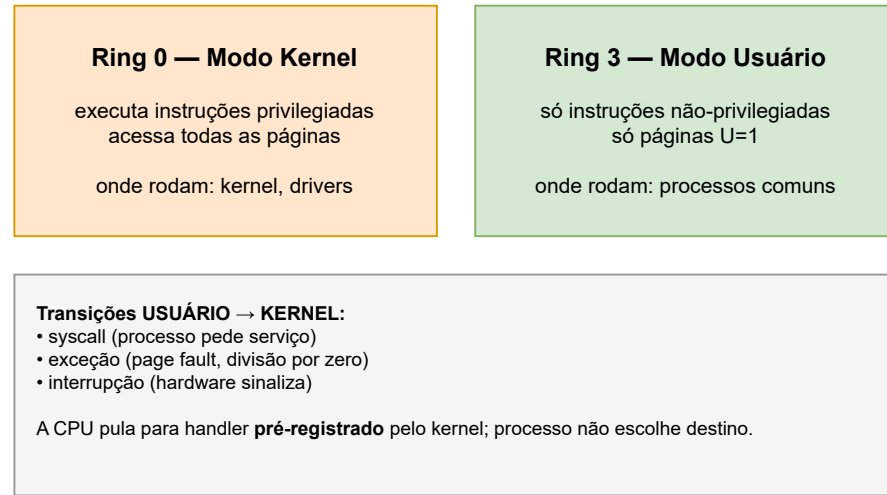


2. PERMISSÕES POR PÁGINA (bits da PTE)

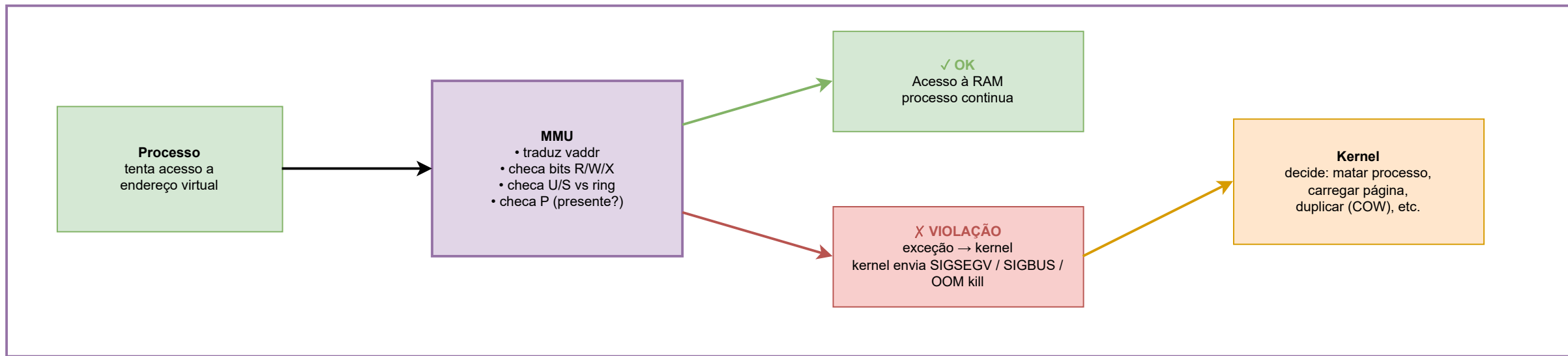
Cada acesso é verificado em hardware pela MMU

P — Present	página em RAM? P=0 → page fault
R/W — Read/Write	permite escrita? tentativa em página R-O → SIGSEGV
U/S — User/Super	usuário pode acessar? U=0 + ring 3 → SIGSEGV
NX — No Execute	página executável? NX=1 + execução → SIGSEGV (anti-exploit)

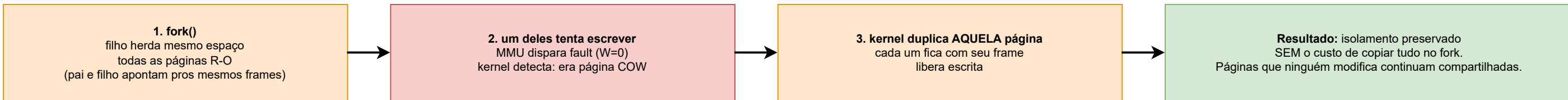
3. MODOS DE EXECUÇÃO (rings)



MMU = Guarda de Segurança em Hardware



Bonus: Copy-on-Write (COW)



ONDE A PROTEÇÃO PEGA (erros comuns do processo)

- Escrita em .text (R-O) → SIGSEGV (W=0)
- Execução de heap/data → SIGSEGV (NX=1)
- Acesso a página não-mapeada → SIGSEGV (P=0 fora de VMA)
- Acesso à área do kernel → SIGSEGV (U=0)
- Stack overflow → SIGSEGV (guard page invadida)
- Memória mapeada de arquivo truncado → SIGBUS

MITIGAÇÕES EM HARDWARE

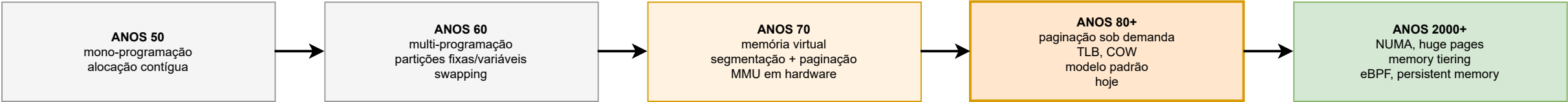
- NX (No Execute) — pages de dados não-executáveis
- SMAP / SMEP — kernel não acessa páginas de usuário sem flag
- ASLR — randomização do layout a cada execução
- KPTI — tabelas separadas user/kernel (pós-Meltdown)
- IOMMU — proteção contra DMA attacks
- ECC — detecção de corrupção em RAM

Quadro Comparativo — Técnicas de Gerenciamento de Memória

Cada técnica resolveu o problema da anterior — e introduziu outro

TÉCNICA	DEFINIÇÃO	PROBLEMA QUE RESOLVE	VANTAGEM	DESVANTAGEM	LIMITAÇÃO PRÁTICA
1. Alocação contígua (anos 50)	Cada processo ocupa um único bloco contíguo de RAM. Proteção via registradores base + limite.	Simplicidade de implementação em sistemas mono-programados.	Acesso direto sem indireção. Relocação simples.	Inflexível: processo cresce e não cabe; outro processo bloqueia espaço adjacente.	Inutilizável para multiprogramação real.
2. Partições fixas (anos 60)	RAM dividida em N partições de tamanho pré-definido em boot; cada processo ocupa uma.	Permitir vários processos simultaneamente.	Implementação simples (tabela fixa). Proteção fácil.	Fragmentação interna grave: processo menor que a partição desperdiça o resto.	Tamanho escolhido em boot — sem flexibilidade para cargas variadas.
3. Partições variáveis (anos 60)	Partições criadas/destruídas dinamicamente conforme processos chegam e saem.	Eliminar fragmentação interna das partições fixas.	Cada processo recebe exatamente o que precisa.	Fragmentação externa: buracos pequenos entre processos que não cabem nada novo.	Exige compactação periódica (cara) ou first/best/worst-fit.
4. Swapping (anos 60–70)	Processos inteiros movidos entre RAM e disco quando há pressão de memória.	Rodar mais processos do que cabem em RAM.	Aumenta número de processos suportados.	Trocar processo inteiro custa caro (segundos); processo fica todo parado durante swap.	Sozinho não escala. Hoje, mecanismo combinado com paginação (swap por página).
5. Paginação ★ MODELO ATUAL ★ (anos 70+)	Espaço virtual e físico divididos em blocos de tamanho fixo (páginas/frames). Qualquer página vai em qualquer frame, via tabela hierárquica.	Eliminar fragmentação externa. Permitir alocação não-contígua e memória virtual.	Sem fragmentação externa. Permite memória virtual, swap por página, proteção fina, COW, sharing.	Pequena fragmentação interna (última página). Overhead de tabela e TLB miss.	Exige MMU em hardware. Tabelas grandes precisam ser hierárquicas.
6. Segmentação (anos 70)	Memória vista como conjunto de segmentos lógicos (código, dados, pilha) de tamanho variável, cada um com base+limite.	Refletir a estrutura lógica do programa no layout de memória.	Compartilhamento e proteção naturais por segmento.	Fragmentação externa entre segmentos. Lógica complexa.	Abandonada como técnica única. Combinada com paginação em alguns x86 antigos.

EVOLUÇÃO HISTÓRICA



Padrão da evolução: cada técnica resolveu uma fragmentação ou rigidez da anterior. A paginação venceu porque trocou fragmentação externa (grave, exige compactação) por fragmentação interna (pequena, previsível, limitada à última página) — e abriu o caminho para memória virtual, COW, swap fino, mmap e tudo o que a Seção 9 mostra que o Linux faz hoje.

Fragmentação — Interna vs Externa

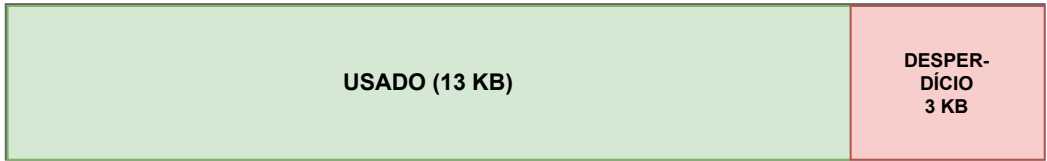
Todo desperdício de memória causado como efeito colateral das estratégias de alocação

FRAGMENTAÇÃO INTERNA

Desperdício DENTRO de um bloco já alocado

O alocador entregou mais espaço do que o processo realmente usa. O resto fica reservado mas inativo.

Exemplo: pedido de 13 KB em página de 16 KB



Página de 16 KB — alocada inteira para o processo. Os 3 KB não-utilizados pertencem a ele, mas não podem ser dados a outro processo.

FRAGMENTAÇÃO EXTERNA

Espaços livres pequenos e ESPALHADOS entre blocos alocados

Memória total livre é grande, mas não há um bloco contíguo grande o suficiente para uma alocação grande.

Exemplo: 100 MB livres, mas pedido de 40 MB falha



Livre total: 45 MB | Maior bloco contíguo: 25 MB | Pedido de 40 MB FALHA.

COMPARAÇÃO LADO A LADO

Aspecto	Interna	Externa
Onde	Dentro de blocos já alocados	Entre blocos livres (espalhados pela memória)
Causa	Alocador entrega mais espaço que o pedido	Alocação/liberação sucessivas deixam buracos
Onde aparece	Partições fixas, paginação (última pág), alocadores com size classes	Partições variáveis, segmentação, vmalloc, huge pages, DMA contíguo
Mitigação	Páginas menores; alocadores que ajustam ao pedido	Paginação (elimina req. de contiguidade) + compactação
Custo da mitigação	Mais overhead de tabela (páginas menores → mais entradas)	Compactação é caríssima; paginação exige MMU

A paginação não elimina fragmentação. Ela troca fragmentação externa (grave, imprevisível, exige compactação) por fragmentação interna (pequena, previsível, limitada à última página).

Esse trade-off é uma vitória clara: o custo da paginação cresce devagar (overhead da tabela) enquanto o benefício cresce rápido (memória virtual, swap por página, COW, sharing).

ALGORITMOS DE ALOCAÇÃO CLÁSSICOS

First-fit

Primeiro buraco que serve
Rápido • fragmenta o início

Best-fit

Menor buraco que serve
Deixa buracos minúsculos inúteis

Worst-fit

Maior buraco, divide
Em teoria evita pequenos • na prática, ruim

Buddy system

Potências de 2; merge de irmãos
Linux usa para frames físicos

Slab

Pools de tamanho fixo
Linux usa para objetos do kernel

Paginação e Memória Virtual — Visão Conceitual

Páginas (virtuais) ↔ Quadros/Frames (físicos) com mapeamento arbitrário via tabela de páginas

MEMÓRIA VIRTUAL DO PROCESSO (contígua, lógica)

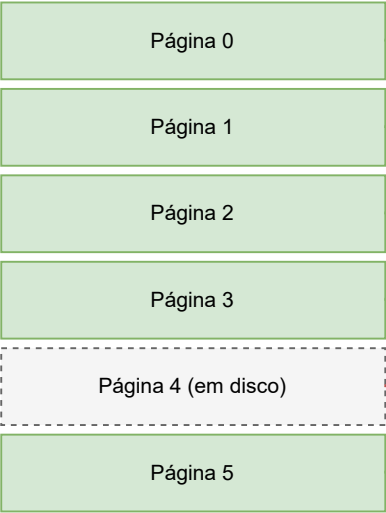
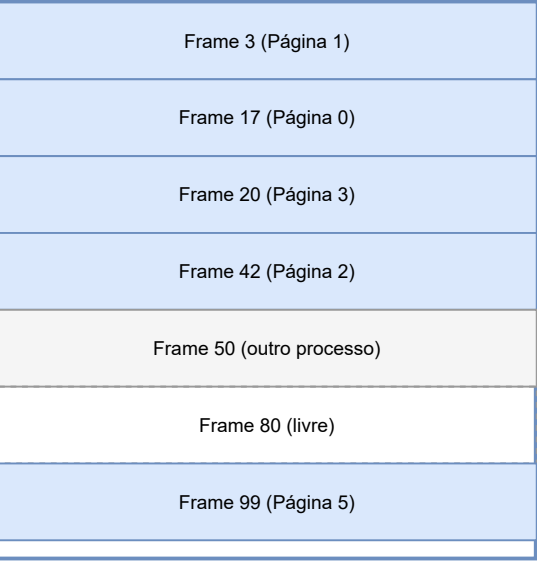


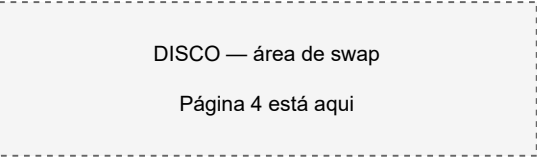
TABELA DE PÁGINAS (hierárquica, mantida pelo kernel)

Página Virt.	Frame	P	R/W	U/S	A	D	NX
0	17	1	0	1	1	0	0 (-text)
1	3	1	1	1	1	1	1 (data)
2	42	1	1	1	0	0	1
3	20	1	1	1	1	1	1
4	(disco)	0	--- página em swap ---				
5	99	1	1	1	0	0	1

MEMÓRIA FÍSICA (RAM) (frames espalhados)



CR3 (registrador da CPU)
aponta para a raiz da tabela
(trocado em cada context switch)



Acesso à Página 4 → PAGE FAULT

- MMU verifica PTE: P=0 → exceção. Kernel detecta tipo:
- major fault: traz a página da área de swap para um frame livre
 - atualiza PTE (P=1, frame correto)
 - invalida TLB
 - retoma instrução (que agora terá sucesso)

TABELA HIERÁRQUICA (x86-64, 4 níveis)

Endereço virtual de 48 bits dividido em 4 índices de 9 bits + offset de 12 bits.

CR3 → PGD → PUD → PMD → PTE → Frame + Offset

Cada nível: 512 entradas (8 bytes × 512 = 4 KiB = uma página).
Tabelas vazias NÃO são criadas — economiza muito espaço.
CPUs novas têm 5 níveis (PML5) cobrindo 128 PiB.

SUBSTITUIÇÃO DE PÁGINA (quando RAM enche)

FIFO
mais antiga sai
ruim

LRU
menos recentemente usada
ótimo • impl. cara

Clock / LRU aprox
bit A + ponteiro circular
usado na prática

Linux: 2 listas LRU
active + inactive
kswapd faz reclaim

Working Set
páginas usadas no intervalo
teórico

LFU
menos frequente
sofre mudança de fase

THRASHING — o ponto de colapso

Quando o working set total dos processos excede a RAM, o kernel:
1) manda página A para swap | 2) processo precisa de A → traz de volta | 3) para trazer, manda B para swap | 4) processo precisa de B → traz...
Resultado: sistema passa mais tempo paginando do que executando. CPU baixa, disco 100%, processos congelados. **Solução:** matar processos (OOM killer) ou adicionar RAM.

Como o Linux Gerencia Memória na Prática

Zonas, alocadores, VMAs, page cache, parâmetros VM e ciclo de vida do processo

1. ZONAS DE MEMÓRIA FÍSICA

ZONE_DMA 0 – 16 MB legacy DMA (24 bits)	ZONE_DMA32 16 MB – 4 GB DMA 32 bits	ZONE_NORMAL acima de 4 GB uso geral	ZONE_MOVABLE configurável huge pages, hotplug
--	--	--	--

Em máquinas NUMA, cada nó tem seu conjunto de zonas. Kernel tenta alocar páginas no nó local. Observar: `/proc/zoneinfo` e `numactl --hardware`

3. VMAs — Virtual Memory Areas

Cada processo tem uma lista de **vm_area_struct** — uma para cada faixa contígua com mesmas permissões e mesmo backing.

Quando ocorre page fault, kernel localiza a VMA que contém o endereço e despacha o handler.

Cada linha de `/proc/maps` corresponde a uma VMA. Campos: `vm_start`, `vm_end`, `vm_flags` (R/W/X/SHARED), `vm_file`, `vm_pgoff`, `vm_ops`.

5. /proc/sys/vm/ — Parâmetros do Subsistema VM

Parâmetro	Função	Default
swappiness	tendência de mandar páginas anônimas para swap (0–100)	60
overcommit_memory	0=heurística, 1=sempre permitir, 2=estricto	0
dirty_ratio	% de RAM até bloquear escritas (writeback síncrono)	20
min_free_kbytes	memória mínima livre que kswapd mantém	calc.
panic_on_oom	pânico do kernel ou só mata processo em OOM	0

7. OOM KILLER

Quando RAM e **swap** esgotam, kernel não consegue mais entregar páginas. Em vez de travar, dispara o **OOM Killer**:

- Cada processo tem um score em `/proc/oom_score` (baseado em uso, idade, prioridade)
- Score ajustável por `/proc/oom_score_adj` (-1000 = imune)
- Processo de maior score recebe **SIGKILL**
- Logado em **dmesg**: Out of memory: Killed process

2. ALOCADORES

Para o kernel

Buddy allocator — base de tudo, aloca em potências de 2 de páginas contíguas

Slab / SLUB — objetos pequenos do kernel (inode, task_struct)

kmalloc — fisicamente contíguo, pequeno

vmalloc — virtualmente contíguo, fisicamente espalhado

Para processos (userspace)

brk / sbrk — move topo do heap (legacy)

mmap — cria nova VMA (arquivo ou anônima)

munmap — remove mapeamento

madvise — dicas ao kernel (sequencial, descarte)

4. PAGE CACHE — o coração do Linux

Aplicação



read/write/mmap

PAGE CACHE
cache de páginas de arquivos em RAM



miss / flush

Disco / FS

- Hit em cache: µs | Miss: ms
- mmap usa diretamente o page cache (sem cópia)
- Aparece como **buff/cache** no free
- **Elástico**: cede sob pressão de memória

6. CICLO DE VIDA DO PROCESSO

fork()

- Nova task_struct
- Tabelas duplicadas, apontando aos **mesmos frames físicos**
- Todas as páginas marcadas R-O
- COW: na 1ª escrita, kernel duplica AQUELA página

exec("/path/programa")

- VMAs construídas do ELF
- VMAs antigas liberadas
- Espaço virtual zerado
- NENHUMA página carregada (demand paging)
- 1º acesso ao .text → page fault, kernel lê do ELF

exit() — Quando o processo termina

- Kernel percorre as VMAs e desfaz cada mapeamento
- Páginas **anônimas** (heap, stack) → buddy allocator
- Páginas de arquivo (libs .so, .text) → page cache (outros processos podem usar)
- Tabelas de páginas liberadas | task_struct liberada via slab

Observabilidade e Monitoramento de Memória no Linux

Como validar o uso da memória — cada ferramenta mostra uma faceta diferente

free

Escopo: sistema inteiro

Mostra: RAM total/usada/livre, buff/cache, swap, **available**

Quando usar: resposta rápida para "quanto de RAM o sistema tá usando?"

Armadilha: olhar "free" baixo e achar que está sem memória. O número que importa é **available**.

top / htop

Escopo: processos

Mostra: lista interativa: VIRT, RES, SHR, %MEM

Quando usar: "quem está consumindo memória?"

Tecla M no top ordena por %MEM. htop tem cores e árvore.

Armadilha: somar RES de todos os processos passa do total da RAM, pois libs compartilhadas são contadas N vezes.

vmstat

Escopo: sistema (séries temporais)

Mostra: paginação (si/so), swap, I/O, CPU em intervalos

Quando usar: "há paginação anormal? swap-in/out?"

Colunas-chave: **si** (swap-in), **so** (swap-out), **wa** (I/O wait CPU)

Alerta: si/so > 0 sustentado + wa alto = **thrashing**

/proc/meminfo

Escopo: sistema (painel detalhado)

Mostra: ~50 contadores: Cached, AnonPages, Mapped, Slab, Dirty, Writeback, SwapCached, etc.

Quando usar: diagnóstico fino, separar tipos de uso

Cuidado: muitos campos exigem leitura cuidadosa. Não somar tudo pra obter total — métricas se sobrepõem.

/proc/maps

Escopo: um processo específico

Mostra: cada VMA — faixa, permissões, arquivo, offset, inode

Quando usar: "o que está mapeado no espaço desse processo?"

Caminhos especiais: **[heap]**, **[stack]**, **[vdso]**, **[vvar]**

Ex: 00400000-00452000 r-xp 00000000 fd:00 12345 /usr/bin/cat

/proc/smaps

Escopo: um processo específico (detalhado)

Mostra: maps + Size, **RSS**, **PSS**, Shared/Private Clean/Dirty, **Swap** por VMA

Quando usar: "quanto desse mapeamento está realmente em RAM?"

PSS é a métrica mais honesta: cada página compartilhada é dividida entre os N processos que a usam.

Outras ferramentas úteis

pmap — versão amigável do maps

pidstat -r — page faults por processo (séries)

sar -B — histórico de paginação
sar -r — histórico de uso de RAM

perf stat -e dTLB-load-misses — TLB misses

bpfftrace, eBPF — probes customizados

dmesg — eventos do kernel (incluindo OOM kill)

RECEITAS PRÁTICAS DE DIAGNÓSTICO

Pergunta / Sintoma	Comando
Quanto de memória sobra?	free -h (olhe a coluna available)
Quem tá comendo memória?	top (tecla M para ordenar por %MEM) ou htop
Esse processo, o que tem mapeado?	cat /proc/maps ou pmap
Quanto esse processo usa de verdade (sem dupla contagem)?	grep Pss /proc/smaps awk '{sum+=\$2} END {print sum, "kB"}'
Sistema lento, será swap?	vmstat 1 (observe si e so)
Quem está paginando muito?	pidstat -r 1
Histórico de paginação ao longo do tempo	sar -B (page faults) sar -r (uso de RAM)
OOM matou alguém?	dmesg grep -i "out of memory" ou journalctl -k grep oom

ARMADILHAS COMUNS DE INTERPRETAÇÃO

- "**Memória usada está em 90%**" — page cache conta como used. O número que importa é **available**
- "**Esse processo consome 500 MB (RES)**" — inclui libs compartilhadas. Use **PSS** em smaps
- "**Tem swap usado, sistema está paginando**" — pode ser swap histórico. Olhe **si/so** atual
- "**Vou liberar cache com drop_caches**" — DESACELERA: próximas leituras irão a disco
- "**OOM matou o errado**" — escolha pelo oom_score, ajustável por oom_score_adj

Conclusão Crítica — O Modelo Convergente dos Sistemas Operacionais Modernos

Qual modelo de gerenciamento de memória melhor representa os SOs modernos e por quê?

A RESPOSTA

O modelo dominante é a **paginação por demanda com memória virtual, proteção por hardware, gerenciamento pelo kernel, uso intenso de page cache e observabilidade rica via interfaces do sistema**.

Esse modelo não é uma escolha entre opções equivalentes — é o resultado convergente de 60 anos de evolução em que cada técnica resolveu o problema da anterior, até sobrar um conjunto mínimo, robusto e adaptável.

OS 6 COMPONENTES INTERDEPENDENTES DO MODELO

1. MEMÓRIA VIRTUAL

Processo nunca toca endereço físico. Toda referência é virtual, traduzida pela MMU.

Entrega: isolamento; espaço uniforme por processo; memória maior que RAM via swap; ASLR; sharing eficiente.

2. PAGINAÇÃO

Espaço dividido em páginas de tamanho fixo (4 KiB). Qualquer página vai em qualquer frame.

Entrega: sem fragmentação externa; granularidade fina para swap; COW prático; permissões por página.

3. PROTEÇÃO POR HARDWARE

MMU verifica bits da PTE em paralelo com a tradução. Custo zero quando tudo OK, exceção quando viola.

Entrega: inviolabilidade; modos ring 0/3; mecanismo claro de page fault.

4. GERENCIAMENTO PELO KERNEL

Única entidade que vê o todo. Decide quem cresce, quem encolhe, quem vai pra swap, quem é morto.

Entrega: visão global; políticas únicas; mediação de syscalls; atendimento de faults.

5. PAGE CACHE

RAM ociosa transparente como cache do sistema de arquivos. Cede sob pressão.

Entrega: read elástico; write deferred; unifica read() e mmap(); compartilhado entre processos.

6. OBSERVABILIDADE

Não é decoração — é parte essencial do design. /proc, /sys, free, vmstat, eBPF.

Entrega: diagnóstico imediato; granularidade certa; padronização; extensibilidade.

OS 6 COMPONENTES FORMAM UMA ARQUITETURA, NÃO 6 OPÇÕES SOLTAS

Memória Virtual requer → **Paginação** depende de → **MMU/HW** atendida por → **Kernel** alavanca → **Page Cache** exposta por → **Observabilidade**

Retirar qualquer um quebra os outros. A memória virtual sem paginação não escala. A paginação sem MMU não tem proteção. A MMU sem kernel não tem como ser configurada. O kernel sem page cache desperdiça RAM. O page cache sem observabilidade vira caixa preta. O modelo é uma arquitetura, não uma coleção de features.

Os 4 requisitos inegociáveis que esse modelo entrega simultaneamente:

- ✓ **ISOLAMENTO** entre processos
- ✓ **EFICIÊNCIA** sob pressão de RAM
- ✓ **FLEXIBILIDADE** de layout (heap dinâmico, mmap, COW)
- ✓ **CAPACIDADE DE DIAGNÓSTICO** granular

O QUE PODE MUDAR NO FUTURO

- **Persistent memory** (Optane, CXL) → distinção RAM/disco esmaece, DAX e memory tiering
- **Memória em cluster** (CXL.mem) → frames físicos em outro chassi
- **eBPF para gerência** → políticas customizadas em userspace
- **Capability-based memory** (CHERI) → bits de permissão viram capabilities tipadas

Mas o esqueleto — virtual, paginado, protegido em HW, gerenciado por kernel — não tem alternativa viável.

O modelo que melhor representa sistemas operacionais modernos é **memória virtual paginada com proteção por MMU, gerenciada pelo kernel, otimizada por page cache e instrumentada por interfaces do sistema operacional**. Ele venceu porque foi o único que conseguiu, simultaneamente, oferecer isolamento entre processos, eficiência sob pressão de RAM, flexibilidade de layout e capacidade de diagnóstico — quatro requisitos que sistemas modernos não podem dispensar.

System Design da Memória em Sistemas Operacionais

Texto de Apoio

Gean Feitosa Cavalcante

2026

1. Introdução e enquadramento do problema

A memória é o recurso que sustenta toda execução em um sistema computacional. Sem ela não há instruções para a CPU buscar, nem espaço para os dados manipulados pelos processos. Apesar dessa centralidade, é também o subsistema mais difícil de raciocinar: opera em escalas que vão de nanossegundos (cache) a milissegundos (swap), serve simultaneamente o hardware, o kernel e o usuário, e exige isolamento absoluto entre processos sem perder eficiência.

Este trabalho responde à pergunta central do enunciado — *“como a memória é usada do início ao fim por hardware, processador, kernel, sistema operacional e processos, garantindo desempenho, isolamento, proteção e continuidade da execução?”* — por meio de um System Design composto por onze diagramas visuais e três análises críticas. A escolha de tratar memória como um *sistema* (em vez de “RAM”) é deliberada: só uma visão integrada deixa claro que cada camada existe para resolver um problema das outras.

2. Decisões do projeto

2.1 Estrutura em camadas

O diagrama macro segue a cadeia exata pedida no escopo: **Hardware → CPU → MMU/TLB → Kernel → Gerência de Memória do SO → Processos → Ferramentas de Observação**. Essa ordem foi mantida como esqueleto narrativo do trabalho inteiro, porque ela já reflete o “caminho de um acesso” — entendendo a cadeia, o leitor compreende todas as outras seções.

2.2 Convenção de cinco campos por bloco

Cada bloco visual carrega cinco informações: **nome, função, entrada, saída, risco/limitação**. Essa convenção, exigida pelo escopo, força a tratar cada componente como uma “caixa preta com interface” — abordagem típica de documentação de arquitetura de sistemas. Em particular, o campo

risco evita que o trabalho se resuma a descrever o “caminho feliz” e cobra mostrar como cada peça pode falhar.

2.3 Paleta de cores semântica

Foram adotadas as seis cores sugeridas pelo escopo (azul para hardware/CPU, roxo para tradução, verde para userspace, vermelho para proteção/falha, laranja para kernel, cinza para secundário/swap). A paleta funciona como uma **legenda implícita**: ao olhar qualquer diagrama, o leitor identifica imediatamente que tipo de componente é cada bloco.

2.4 Conteúdo modular reaproveitável

Cada arquivo `.md` da pasta `conteudo/` cumpre duplo papel: alimenta os blocos do design visual e fornece a base teórica resumida no texto de apoio. Esse encadeamento foi escolhido para evitar redundância: a teoria sustenta o visual, e o visual sintetiza a teoria.

3. Conceitos centrais utilizados

Quatro conceitos atravessam o trabalho inteiro:

Memória virtual — o processo nunca toca o endereço físico; cada acesso passa pela MMU. Esse fato isolado libera proteção, *sharing* e *swap*, e é a base sobre a qual tudo se monta. Sem memória virtual, isolamento entre processos seria impossível em sistemas multiusuário.

Paginação — espaço virtual e físico divididos em blocos de tamanho fixo (4 KiB no Linux x86-64). Resolve fragmentação externa, permite alocação não-contígua, suporta *swap* por granularidade fina, e torna o *Copy-on-Write* prático. As seções 6, 7 e 8 do conteúdo examinam, respectivamente, por que ela venceu a alocação contígua, qual fragmentação ela introduz no lugar da externa, e como funciona em detalhe.

Proteção por hardware — a MMU verifica os bits da PTE em cada acesso. Sem essa verificação ser feita em silício, o custo seria proibitivo. A seção 5 mostra como essa proteção, somada à separação de modos usuário/kernel, é o que torna possível multiusuário moderno.

Page cache — a grande “invenção operacional” do Linux moderno: usar a RAM ociosa como cache transparente do sistema de arquivos. A seção 9 mostra que esta é, na prática, a maior consumidora de RAM em um Linux saudável — e que é elástica, cede sob pressão. Ignorar isso leva ao erro clássico do operador ao olhar `free` e achar que o sistema está “sem memória”.

4. Relação entre visual e teoria

Cada diagrama do design tem correspondência direta com uma seção do conteúdo escrito:

Diagrama visual	Conteúdo escrito
01 — Diagrama Macro	Seção 1: cadeia HW→observação
02 — Hierarquia de Memórias	Seção 2: registradores → cache → RAM → swap
03 — Fluxo de Tradução	Seção 3: passo a passo do acesso
04 — Mapa do Processo	Seção 4: texto, dados, heap, stack, mmap
05 — Proteção e Isolamento	Seção 5: PTE, modos kernel/usuário, COW
06 — Técnicas de Gerenciamento	Seção 6: seis técnicas comparadas
07 — Fragmentação	Seção 7: interna vs externa
08 — Paginação Visual	Seção 8: páginas, frames, page fault
09 — Linux na Prática	Seção 9: zonas, page cache, OOM
10 — Observabilidade	Seção 10: free, top, vmstat, /proc
11 — Conclusão Visual	Análise crítica e síntese

As três análises críticas (*trade-offs*, erros e riscos, conclusão crítica) não têm diagrama dedicado mas amarram visualmente os blocos anteriores: o *trade-off* “contígua vs paginação” referencia os diagramas 6 e 7; “*swap* excessivo” da análise de riscos cita o diagrama 3 de fluxo.

5. Como o Linux comprova na prática

Um dos principais argumentos do trabalho é que **o modelo proposto não é teórico**: ele está implementado, observável e mensurável em qualquer Linux. As correspondências concretas:

- **Tradução de endereço (diagrama 3)** — verificável em `perf stat -e dTLB-load-misses` durante a execução de uma aplicação
- **Mapa do processo (diagrama 4)** — visível em tempo real em `cat /proc/<pid>/maps` para qualquer PID
- **Bits de permissão (diagrama 5)** — refletidos nas colunas `r/w/x/p` do mesmo arquivo
- **Paginação sob demanda (diagrama 8)** — comprovável: ao executar um programa grande, `mmap -X <pid>` mostra que `RSS << VIRT` no início, e cresce conforme uso
- **Page cache (diagrama 9)** — observável: rodar `cat /grande_arquivo > /dev/null` aumenta `Cached` em `/proc/meminfo`
- **Swap (diagrama 9)** — disparável: alocar memória até estourar e ver `vmstat 1` reportar `si/so > 0`
- **OOM killer (diagrama 9)** — visível em `dmesg` após teste de pressão

Cada conceito do design tem um experimento simples que o demonstra. O trabalho não é uma descrição de “como deveria ser” — é uma descrição de “como é”.

6. Trade-offs e análise crítica

Toda decisão de design de memória resolve um problema e **cria outro**. Cinco tensões fundamentais foram analisadas no conteúdo detalhado:

1. **Desempenho vs Proteção** — o Linux sempre fica do lado da proteção, mesmo quando isso custa centenas de ciclos em cada syscall ou troca de contexto.
2. **Simplicidade vs Flexibilidade** — flexibilidade vence, ao custo de milhares de páginas de documentação técnica. A alternativa seria não rodar cargas reais.
3. **Memória Contígua vs Paginação** — paginação venceu há quatro décadas. Apenas DMA legado, *huge pages* e NUMA local ainda exigem contiguidade.
4. **Menos Fragmentação vs Maior Overhead** — Linux usa páginas de 4 KiB por padrão, com *Transparent Huge Pages* automáticas para regiões grandes.
5. **Observabilidade Simples vs Detalhada** — começar com `free` e `top`; ir fundo apenas quando o básico não explica.

Os principais riscos operacionais identificados foram: fragmentação (interna e externa), *swap* excessivo (*thrashing*), overhead de paginação por *TLB miss*, mau uso de cache (`drop_caches` em produção), falhas de proteção, crescimento descontrolado de *heap*, *stack overflow* e leitura incorreta de métricas (confundir `buff/cache` com memória perdida).

7. Resposta à pergunta central

A pergunta do escopo é respondida em três níveis:

No nível mais concreto (diagrama 3): um acesso à memória começa com um endereço virtual emitido pela CPU; passa pela MMU para tradução; consulta a TLB e, se preciso, a tabela de páginas; resulta em endereço físico ou em *page fault* tratado pelo kernel. Esse ciclo se repete bilhões de vezes por segundo.

No nível arquitetural (diagrama 1 e conclusão crítica): a memória é gerenciada por uma cadeia de camadas com responsabilidades distintas — hardware fornece o substrato, MMU traduz e protege, kernel coordena, processos consomem, ferramentas observam. Cada camada existe porque resolve um problema da anterior.

No nível operacional (diagramas 9 e 10): no Linux, isso se concretiza em zonas, *page cache*, *swap*, VMAs, paginação sob demanda, e em ferramentas que expõem cada métrica relevante ao operador.

O sistema é construído para ser *visto* — toda decisão importante é refletida em `/proc` ou em sinais.

8. Conclusão

O trabalho mostra que memória, em sistemas modernos, não é “RAM gerenciada” — é um **sistema** com responsabilidades distribuídas entre hardware, kernel, processos e ferramentas. O modelo que melhor representa SOs modernos combina memória virtual, paginação, proteção em hardware, gerenciamento centralizado pelo kernel, *page cache* elástico e observabilidade rica. Esse modelo venceu porque é o único que entrega simultaneamente isolamento, eficiência, flexibilidade e capacidade de diagnóstico — quatro requisitos inegociáveis em qualquer sistema multiusuário atual.

O design visual deste trabalho não tenta inventar uma representação nova — ele **organiza visualmente o que o Linux já implementa**, conectando teoria, diagrama e ferramenta de modo que cada peça seja localizável tanto em livros de Sistemas Operacionais quanto em uma sessão de terminal.

Referências

System Design da Memória em Sistemas Operacionais

Gean Feitosa Cavalcante

2026

Referências

Documentação oficial do Linux Kernel

KERNEL.ORG. **Page Tables — The Linux Kernel documentation**. Disponível em: https://docs.kernel.org/mm/page_tables.html. Acesso em: 2026.

KERNEL.ORG. **Memory Management — The Linux Kernel documentation**. Disponível em: <https://docs.kernel.org/admin-guide/mm/index.html>. Acesso em: 2026.

Manuais do sistema (man pages)

KERRISK, Michael. **proc_pid_maps(5) — Linux manual page**. Disponível em: https://man7.org/linux/man-pages/man5/proc_pid_maps.5.html. Acesso em: 2026.

KERRISK, Michael. **proc_pid_smaps(5) — Linux manual page**. Disponível em: https://man7.org/linux/man-pages/man5/proc_pid_smaps.5.html. Acesso em: 2026.

KERRISK, Michael. **proc(5) — Linux manual page**. Disponível em: <https://man7.org/linux/man-pages/man5/proc.5.html>. Acesso em: 2026.

KERRISK, Michael. **vmstat(8) — Linux manual page**. Disponível em: <https://man7.org/linux/man-pages/man8/vmstat.8.html>. Acesso em: 2026.

KERRISK, Michael. **free(1) — Linux manual page**. Disponível em: <https://man7.org/linux/man-pages/man1/free.1.html>. Acesso em: 2026.

KERRISK, Michael. **top(1) — Linux manual page**. Disponível em: <https://man7.org/linux/man-pages/man1/top.1.html>. Acesso em: 2026.

Artigos acadêmicos

CARLSON, T. E. et al. **What Do You Mean by Memory? When Engineers Are Lost in the Maze of Complexity**. Proceedings of the ACM. 2024. Disponível em: <https://dl.acm.org/doi/pdf/10.1145/3639477.3639735>. Acesso em: 2026.

KANTREIS, M. et al. **eBPF-mm: Userspace-guided memory management in Linux with eBPF**. arXiv preprint. 2024. Disponível em: <https://arxiv.org/html/2409.11220v1>. Acesso em: 2026.

AMIT, N.; TAI, A. **Skip TLB flushes for reused pages within mmap's**. arXiv preprint. 2024. Disponível em: <https://arxiv.org/pdf/2409.10946.pdf>. Acesso em: 2026.

WU, J. et al. **Non-volatile main memory management methods based on a file system**. PubMed Central. Disponível em: <https://pmc.ncbi.nlm.nih.gov/articles/PMC4162891/>. Acesso em: 2026.

Observação sobre as fontes

As referências de **documentação oficial** (kernel.org e man7) sustentam todas as partes técnicas e práticas do trabalho — especialmente as seções sobre MMU, tabelas de páginas, `/proc/<pid>/maps`, `/proc/<pid>/smaps`, `vmstat` e o subsistema VM do Linux.

Os **artigos acadêmicos** foram utilizados para fortalecer a análise crítica, os *trade-offs* e a discussão sobre tendências modernas em gerenciamento de memória (eBPF, *TLB flush optimization*, memória não-volátil).